Instructions for the Laboratory Session

Analysis and Design of Algorithms (ADA)

The purpose of this laboratory is not simply to write and execute 25 programs. The objective is to help you develop the ability to **understand a problem, design an algorithm, implement the solution, analyse its efficiency, test it, and explain it confidently**.

You are therefore expected to learn every program rather than simply completing the laboratory record.

1. Understand the Purpose of the Laboratory

In this laboratory, you will work with a set of 25 programs based on the concepts covered in the ADA course.

For every program, you are expected to understand:

- What problem does the program solve?
- Why is this problem important?
- What is the algorithm used?
- How does the algorithm work?
- What are the steps involved?
- How is the algorithm implemented in code?
- How efficient is the algorithm?
- What are its best, average, and worst cases?
- Where can the algorithm be used in real-world applications?

Remember:

Your objective is not to complete 25 programs. Your objective is to learn 25 algorithms and problem-solving approaches.

2. Do Not Start by Copying the Code

When a new program is given, do not immediately search for the source code.

Follow this sequence:

Understand the Problem → Identify the Algorithm → Write the Steps → Perform a Dry Run → Write the Code → Execute → Test → Analyse → Explain

You should first understand the logic and then write or study the implementation.

3. Every Program Must Have Five Stages

For every laboratory program, you must complete the following five stages.

Stage 1: Understand

Read the problem statement carefully.

Identify:

- Input
- Output
- Constraints
- Expected result
- Algorithmic technique

Stage 2: Design

Before coding, write:

- Algorithm
- Pseudocode, wherever required
- Flowchart, wherever applicable

Stage 3: Implement

Write the program using the prescribed programming language.

Use:

- Meaningful variable names
- Proper indentation
- Comments where necessary
- Modular and readable code

Stage 4: Test and Analyse

Run the program using different inputs.

Check:

- Normal cases
- Boundary cases
- Special cases
- Invalid cases, wherever applicable

Then determine:

- Time complexity
- Space complexity
- Best case

- Average case, wherever applicable
- Worst case

Stage 5: Explain

You must be able to explain your program to the faculty.

You may be asked:

- What does this line do?
- Why did you use this variable?
- What happens if the input changes?
- What is the time complexity?
- What is the space complexity?
- Can you modify the program?
- Can you predict the output for a given input?

4. Maintain Your ADA Lab Record

For every program, maintain the following sections:

1. Program Number
2. Program Title
3. Problem Statement
4. Objective
5. Input
6. Expected Output
7. Algorithm
8. Pseudocode, wherever applicable
9. Flowchart, wherever applicable
10. Dry Run
11. Source Code
12. Test Cases
13. Output Screenshots
14. Time Complexity
15. Space Complexity
16. Applications
17. Learning Reflection
18. Faculty Verification

Your record should demonstrate your learning process.

5. Maintain a Digital Lab Portfolio

In addition to the laboratory record, each student should maintain a digital folder.

Create one main folder:

ADA_LAB_YourName_USN

Inside it, create 25 folders:

01_Program_Name

02_Program_Name

03_Program_Name

Continue until Program 25.

Save your:

- Source code
- Dry runs
- Test cases
- Output screenshots
- Complexity analysis
- Other required documentation

Do not wait until the end of the semester to organize your files.

6. Complete the Program Before Moving to the Next One

A program will be considered **fully completed** only when you have:

- Understood the problem
- Written the algorithm
- Completed the dry run
- Implemented the program
- Executed the program successfully
- Tested the program
- Analysed its complexity
- Explained the program during verification

Do not simply collect 25 source codes.

---

7. Dry Run Is Mandatory

Before executing the program, perform at least one dry run manually.

For example, if the program is sorting an array, take a small input such as:

[5, 2, 8, 1]

Show how the algorithm changes the data at each step.

A dry run helps you understand the actual working of the algorithm.

If you cannot perform a dry run, it is likely that you have not yet understood the algorithm.

8. Do Not Test Only One Input

A program that works for one input is not necessarily a correct program.

For every program, try different types of inputs.

For example:

- Small input
- Large input
- Normal input
- Boundary input
- Empty input, wherever applicable
- Duplicate values, wherever applicable
- Already sorted data
- Reverse sorted data

Record the test cases and results.

9. Learn Time and Space Complexity

ADA is not only about getting the correct output.

You must also understand **how efficiently the solution works**.

For every program, ask:

How much time does the algorithm require as the input size increases?

How much memory does the algorithm require?

You should be familiar with notations such as:

- $O(1)$
- $O(\log n)$
- $O(n)$
- $O(n \log n)$
- $O(n^2)$
- $O(2^n)$

Do not simply memorize the complexity. Understand why the algorithm has that complexity.

10. Faculty Verification

Each program must be verified by the faculty.

During verification, you may be asked to:

- Explain the algorithm.
- Explain your source code.
- Perform a dry run.
- Explain the output.
- Identify the complexity.
- Modify the program.
- Find an error in the program.
- Predict the output for a new input.

Therefore, **do not come for verification without understanding your program**.

11. AI Tools and Online Resources

You may use:

- Textbooks
- Reference books
- Lecture notes
- Online tutorials
- Documentation
- AI-based learning tools

However, these resources should be used for **learning**, not for blindly copying answers.

If you use an AI tool to understand or improve a program, you must still be able to:

- Explain the logic.
- Explain the code.
- Identify the algorithm.
- Explain the complexity.
- Modify the program independently.

The final responsibility for the submitted work is yours.

12. Academic Integrity

The following practices are not acceptable:

- Copying the complete program from another student.
- Submitting a program that you cannot explain.
- Copying output screenshots from another student.
- Claiming to have executed a program that you have not executed.
- Submitting identical documentation without understanding the work.

You may discuss concepts with your classmates, but every student must develop an individual understanding of the program.

13. Ask Questions Without Hesitation

If you do not understand a program, ask.

You are encouraged to ask:

- Why does this algorithm work?
- Why is this step required?
- Why is this complexity obtained?
- What happens if the input changes?
- Is there another way to solve the problem?
- Which approach is more efficient?

There is no penalty for asking questions.

There is, however, a problem if you complete a program without understanding it.

14. Maintain Your Own Progress Tracker

You will receive a list of 25 programs.

Track your progress regularly.

For every program, mark:

- Not Started
- Understanding Completed
- Algorithm Completed
- Dry Run Completed
- Coding Completed
- Execution Completed
- Testing Completed
- Complexity Completed
- Viva Completed
- Fully Verified

Do not wait until the last few laboratory sessions to complete all programs.

15. Your Lab Performance Will Be Based on Learning

Your laboratory performance will not be judged only by the number of programs completed.

You will be evaluated based on:

- Understanding of the algorithm
- Problem-solving ability
- Program implementation
- Correct execution
- Testing ability
- Complexity analysis
- Documentation
- Viva performance
- Ability to modify or adapt the program

The ADA Lab Rule

For every program, remember:

UNDERSTAND → DESIGN → DRY RUN → CODE → EXECUTE → TEST → ANALYSE → EXPLAIN

If you follow these eight steps for all 25 programs, you will not only complete your laboratory work but also develop a strong foundation in algorithmic problem-solving.

Your First Task Today

During today's laboratory session:

Step 1
Open the list of 25 laboratory programs.
Step 2
Read all 25 programs once.
Step 3
Identify the different algorithmic techniques covered.
Step 4
Create your ADA Lab Record.
Step 5
Create your digital ADA Lab Portfolio.
Step 6
Create your 25-program progress tracker.
Step 7

Begin with Program 1.

Step 8

Before writing the code, discuss the problem and algorithm with your faculty.

Step 9

Write the algorithm and perform a dry run.

Step 10

Only then proceed to implementation and execution.

Final Message to Students

Do not measure your success by how quickly you finish the 25 programs. Measure your success by how confidently you can explain and solve a new problem using the concepts you have learned.